

Simulador Hospitalar Multiagente

O objetivo deste trabalho prático é desenvolver uma aplicação visual no ambiente Processing (modo Java) que simule a dinâmica de um setor de atendimento hospitalar. A simulação baseia-se em um modelo multiagente, onde cada paciente é representado como um agente autônomo com objetivos, estados e tomadas de decisão próprias.

Os estudantes deverão integrar diversos conceitos fundamentais de estruturas de dados e algoritmos de busca para dar vida ao hospital, modelado como um grid bidimensional. Ao longo do desenvolvimento, espera-se a aplicação prática de conceitos como alocação dinâmica de memória (estruturas encadeadas), árvores de decisão baseadas em vetores e caminhos mínimos em ambientes discretos.

O Grid do Hospital e o Funcionamento de Cada Bloco

O hospital é representado por uma matriz bidimensional onde cada célula possui um papel fixo de infraestrutura ou de serviço. A movimentação dos pacientes e a dinâmica do atendimento dependem diretamente da lógica de cada bloco do grid:

- **Chão (.):** É a célula transitável padrão. Qualquer paciente pode caminhar livremente por ela. O único impedimento é que duas entidades não podem ocupar a mesma célula de chão ao mesmo tempo.
- **Parede (#):** Representa barreiras físicas intransponíveis. O algoritmo de busca de caminhos deve ignorar estas células, e os personagens nunca podem atravessá-las.
- **Gerador (G):** Há exatamente um no mapa. É o ponto de entrada física na simulação. Novos pacientes surgem nesta coordenada de forma lógica e visual, desde que a célula esteja livre.
- **Removedor (R):** Há exatamente um no mapa. É o ponto de saída. Quando um paciente que já concluiu todo o seu tratamento médico pisa nesta célula, ele é retirado da lista de agentes ativos e suas estatísticas são computadas pelo sistema.
- **Totem (T):** Dispositivo eletrônico onde o paciente retira sua senha de atendimento. O paciente deve se deslocar até o totem logo após entrar no hospital. Apenas um paciente pode interagir com cada totem por vez.
- **Assento (A):** Células onde os pacientes aguardam a sua vez de serem chamados (seja para a triagem ou para o atendimento médico). Um assento possui estados lógicos claros: livre, reservado (quando um paciente está caminhando em direção a ele) e ocupado (quando o paciente está efetivamente sentado). Reservar o assento antes de iniciar o deslocamento impede que dois pacientes caminhem para a mesma cadeira.
- **Enfermeira de Triagem (E):** Posto fixo de atendimento de enfermagem. O paciente não

deve pisar na célula ocupada pela enfermeira. Ele deve se deslocar para uma célula de chão livre adjacente (vizinha) à enfermeira. O atendimento é considerado iniciado quando o paciente chega a essa posição adjacente.

- **Médico (M):** Consultório médico de atendimento. Assim como na triagem, o paciente não sobrepõe a célula do médico; ele se posiciona em uma célula livre adjacente para realizar a consulta.

Fluxo Completo do Paciente e o Sistema de Senhas

A jornada de um paciente dentro da simulação segue um ciclo de vida rígido baseado em estados internos. Ao nascer no Gerador (G), o paciente recebe um identificador único, um tipo de atendimento (gerado com 25% de chance de ser preferencial e 75% de ser normal) e um estado inicial de trânsito.

Seu primeiro destino é o Totem (T). Ao alcançar o totem, o paciente retira uma senha de atendimento. O totem gerencia duas sequências numéricas independentes: uma para senhas normais (ex: N0001, N0002) e outra para preferenciais (ex: P0001, P0002). O código da senha gerada deve ser armazenado no objeto do paciente.

Após retirar a senha, o paciente entra imediatamente na Fila de Triagem correspondente (Fila Normal ou Fila Preferencial) e procura o assento livre mais próximo para sentar e aguardar.

Quando uma enfermeira fica ociosa, ela analisa as duas filas de triagem e seleciona o próximo paciente. Para evitar que a fila normal fique sem atendimento caso haja um fluxo contínuo de preferenciais, o sistema deve adotar uma regra de alternância: atender no máximo 2 pacientes preferenciais consecutivos para cada 1 paciente normal atendido (caso este exista).

Ao ser chamado para a triagem, o paciente libera seu assento, muda seu destino para o posto da enfermeira e se desloca até lá. O tempo de triagem começa a contar apenas quando o paciente atinge a célula adjacente à enfermeira. Terminada a triagem, as características clínicas do paciente são processadas por uma árvore de decisão para definir sua classificação de risco de Manchester.

Com a classificação em mãos (uma cor associada à gravidade), o paciente é inserido em uma das cinco filas de atendimento médico (Fila Vermelha, Laranja, Amarela, Verde ou Azul) e retorna para um assento livre na sala de espera.

Quando um médico fica ocioso, ele busca o próximo paciente respeitando rigorosamente a ordem de gravidade clínica: a Fila Vermelha tem prioridade absoluta sobre a Laranja, que tem sobre a Amarela, e assim por diante. Dentro de uma mesma cor, o atendimento segue a ordem de chegada (FIFO). Após ser chamado pelo médico, o paciente libera o assento, desloca-se até o consultório adjacente ao médico, passa pela consulta e, ao término, recebe a célula do Removedor (R) como destino final para deixar o hospital.

Protocolo de Manchester e a Árvore de Decisão no Vetor

O Protocolo de Manchester é um sistema de triagem médica utilizado no mundo inteiro para categorizar a gravidade do estado de saúde dos pacientes, garantindo que os casos mais críticos sejam atendidos primeiro. Em nossa simulação, cada paciente possui em sua estrutura um arranjo de características clínicas que simulam seus sinais vitais, gerados no momento de sua criação:

- Índice 0: **Saturação de Oxigênio** (valores típicos: 70 a 100)
- Índice 1: **Temperatura Corporal** (valores típicos: 34 a 42)
- Índice 2: **Nível de Dor** (escala inteira de 0 a 10)
- Índice 3: **Consciência Alterada** (0 para não, 1 para sim)

Para classificar o paciente em uma das cinco cores do protocolo, utilizaremos uma árvore de decisão baseada nos sinais vitais acima. Esta árvore deve ser representada de forma estática dentro de um vetor de objetos da classe NoManchester, utilizando o mapeamento clássico de índices de heaps binários.

Para um nó que está na posição i do vetor, seu filho esquerdo (caminho para teste VERDADEIRO) fica no índice $2i + 1$, e seu filho direito (caminho para teste FALSO) fica no índice $2i + 2$. A árvore pedagógica obrigatória do projeto possui a seguinte estrutura lógica:

- **Nó 0 (Raiz):** Verifica se a *Consciência Alterada* é igual a 1.
 - *Verdadeiro (Filho Esquerdo - Nó 1):* **Folha Vermelha** (Emergência).
 - *Falso (Filho Direito - Nó 2):* Avança para o próximo teste.
- **Nó 2:** Verifica se a *Saturação de Oxigênio* é menor que 92.
 - *Verdadeiro (Filho Esquerdo - Nó 5):* **Folha Laranja** (Muito Urgente).
 - *Falso (Filho Direito - Nó 6):* Avança para o próximo teste.
- **Nó 6:** Verifica se o *Nível de Dor* é maior ou igual a 8.
 - *Verdadeiro (Filho Esquerdo - Nó 13):* **Folha Amarela** (Urgente).
 - *Falso (Filho Direito - Nó 14):* Avança para o próximo teste.
- **Nó 14:** Verifica se a *Temperatura Corporal* é maior ou igual a 38.0.
 - *Verdadeiro (Filho Esquerdo - Nó 29):* **Folha Verde** (Pouco Urgente).
 - *Falso (Filho Direito - Nó 30):* **Folha Azul** (Não Urgente).

Os nós intermediários que não são utilizados nesse desenho da árvore devem ser inicializados como nulos ou marcados como inativos no vetor. O algoritmo de classificação consiste em iniciar um ponteiro de leitura no índice 0 do vetor. Enquanto o nó atual não for um nó folha, o sistema extrai o atributo do paciente, aplica a comparação matemática programada naquele nó e atualiza o índice atual para $2i + 1$ ou $2i + 2$ de acordo com o resultado booleano. Ao encontrar um nó marcado como folha, o percurso termina e a cor associada é atribuída ao paciente.

O Algoritmo Wavefront e Busca em Largura (BFS)

Os pacientes não sabem nativamente como chegar aos seus destinos. Para encontrar o caminho mais curto desviando das paredes do hospital, o simulador deve utilizar o algoritmo Wavefront. Ele funciona gerando uma matriz de distâncias inteiras com as mesmas dimensões do mapa. O cálculo começa no destino (que recebe distância zero) e se propaga pelas células vizinhas transitáveis passo a passo.

Uma implementação conceitual do algoritmo em Processing (modo Java) utilizando uma classe de fila encadeada própria para coordenadas é apresentada abaixo:

```
// Classe auxiliar simples para representar coordenadas no grid
class Coordenada {
    int linha;
    int coluna;
    Coordenada(int l, int c) {
        this.linha = l;
        this.coluna = c;
    }
}

// Implementação do Wavefront que retorna a matriz de distâncias
int[][] calcularWavefront(int destinoL, int destinoC, char[][] mapa) {
    int numLinhas = mapa.length;
    int numColunas = mapa[0].length;
    int[][] distancias = new int[numLinhas][numColunas];

    // Inicializa toda a matriz com -1 (indicando células não visitadas)
    for (int i = 0; i < numLinhas; i++) {
        for (int j = 0; j < numColunas; j++) {
            distancias[i][j] = -1;
        }
    }

    // A própria fila coordenada desenvolvida pelo grupo deve ser usada aqui
    FilaCoordenadas fila = new FilaCoordenadas();

    // Define a distância do destino como zero e o insere na fila
    distancias[destinoL][destinoC] = 0;
    fila.enqueue(new Coordenada(destinoL, destinoC));

    // Vetores auxiliares para percorrer os 4 vizinhos (Cima, Baixo, Esquerda, Direita)
    int[] dLinha = {-1, 1, 0, 0};
    int[] dColuna = {0, 0, -1, 1};

    while (!fila.vazia()) {
        Coordenada atual = fila.dequeue();

        for (int i = 0; i < 4; i++) {
```

```

int novoL = atual.linha + dLinha[i];
int novoC = atual.coluna + dColuna[i];

// Verifica se a coordenada vizinha está dentro dos limites do mapa
if (novoL >= 0 && novoL < numLinhas && novoC >= 0 && novoC < numColunas) {
    // Verifica se a célula é transitável (não é parede '#') e ainda não foi visitada
    if (mapa[novoL][novoC] != '#' && distancias[novoL][novoC] == -1) {
        distancias[novoL][novoC] = distancias[atual.linha][atual.coluna] + 1;
        fila.enqueue(new Coordenada(novoL, novoC));
    }
}
}
}
}

return distancias;
}

```

Para caminhar, o paciente que se encontra na posição (L, C) analisa as distâncias das suas quatro células vizinhas na matriz gerada pelo Wavefront. Ele deve escolher a célula vizinha que possua o menor valor numérico maior ou igual a zero. Se a célula de menor valor estiver ocupada por outro paciente naquele instante, ele tenta a segunda menor, e assim sucessivamente. Caso todas as opções estejam bloqueadas, o agente permanece parado no quadro atual.

Temporização da Simulação e Variáveis de Tempo

Para que a simulação seja fluida e rápida o suficiente para fins de teste e apresentação em laboratório, os tempos de spawn (geração de novos agentes) e de atendimento devem ser curtos, porém dinâmicos. Em vez de utilizar intervalos fixos artificiais, os tempos devem ser calculados com variações dinâmicas para simular o comportamento humano real.

Tempo entre Chegadas (Spawn de Pacientes)

Novos pacientes não devem surgir em intervalos idênticos de tempo. Para simular a imprevisibilidade de um hospital real, utilizamos a distribuição exponencial. Matematicamente, a distribuição exponencial é ideal para modelar o tempo entre eventos independentes que ocorrem a uma taxa média constante. Em termos simples, ela faz com que intervalos curtos entre pacientes sejam muito comuns, mas permite que ocorram ocasionalmente intervalos mais longos, evitando a previsibilidade de um metrônomo.

A fórmula para sortear o tempo até a próxima chegada em segundos baseia-se na geração de um número aleatório uniforme u entre 0 e 1 obtido do Processing:

```
float mediaSpawn = 5.0; // Média de 5 segundos
float u = random(0, 1); // Número aleatório uniforme entre 0 e 1
float tempoProximoSpawn = -mediaSpawn * log(1 - u);
```

Para uma simulação dinâmica e ágil no ambiente de avaliação, adote o valor de **media = 5.0 segundos**. Isso significa que, em média, um novo paciente tentará nascer a cada 5 segundos no ponto de geração, com alguns surgindo em apenas 1 segundo e outros demorando até 15 segundos para aparecer.

Tempos de Atendimento (Triagem e Consulta)

A duração dos atendimentos na triagem e nos consultórios médicos segue um comportamento em torno de uma média esperada, com pequenas variações para mais ou para menos. Para isso, utilizamos a distribuição normal (ou gaussiana), que gera a clássica curva em formato de sino. A função nativa `randomGaussian()` do Processing gera valores com média 0 e desvio padrão 1.

Para transformar esse valor em segundos do mundo real, multiplicamos pelo desvio padrão desejado e somamos a média física. Para evitar valores absurdos ou negativos que possam travar os agentes, aplicamos um limite mínimo físico:

```
tempoAtendimento = max(media + (desvio * randomGaussian()), minimo);
```

Os parâmetros fixos obrigatórios para manter a simulação rápida e eficiente são:

- **Tempo de Triagem (Enfermeira):** Média de **6.0 segundos**, Desvio Padrão de **2.0 segundos** e Limite Mínimo de **2.0 segundos**.
- **Tempo de Consulta (Médicos):** Média de **12.0 segundos**, Desvio Padrão de **4.0 segundos** e Limite Mínimo de **4.0 segundos**.

Lógica das Telas e Sistema de Menus

O ciclo de vida da interface gráfica do programa divide-se em três estados bem delineados: Menu Inicial, Simulação Ativa e Menu de Pausa.

No **Menu Inicial**, o foco do usuário está na escolha do cenário. O sistema deve ler a pasta de dados do projeto e exibir na tela os botões de seleção de mapa dinamicamente. O mapa selecionado deve ter destaque visual claro. Ao clicar em "Iniciar Simulação", o sistema carrega o arquivo do mapa escolhido, inicializa as matrizes de ocupação física, cria as instâncias das filas e listas de pacientes vazias e dispara o relógio de simulação.

Durante a **Simulação Ativa**, a interface renderiza o grid, os sprites dos pacientes se movendo e o painel de dados estatísticos. Caso o usuário pressione a tecla ESC ou clique no botão de pausa, a renderização física do hospital é interrompida e o **Menu de Pausa** é sobreposto à tela. No **Menu de Pausa**, as seguintes operações devem ser garantidas:

- **Continuar:** Retoma a simulação exatamente do ponto em que parou, calculando a diferença temporal da pausa para não causar saltos bruscos no relógio interno do hospital.
- **Resetar:** Recarrega o mapa selecionado do zero, limpa todas as filas de prioridades e de triagem, esvazia a lista encadeada de pacientes do hospital, redefine todos os assentos como livres, reinicia o estado dos médicos e enfermeiras para ocioso e zera todas as métricas do painel estatístico, reiniciando o relógio.
- **Voltar ao Menu Inicial:** Limpa os dados da simulação em execução e retorna o usuário à tela de seleção de mapas, permitindo a escolha de um novo cenário sem fechar o programa.

Detalhamento das Entregas e Uso do GitHub

O gerenciamento de código do projeto deve obrigatoriamente ser feito de forma colaborativa e gradual por meio de um repositório no GitHub. O histórico de commits deve refletir a evolução lógica do desenvolvimento do simulador.

Cada commit feito por um integrante do grupo deve conter mensagens claras e profissionais em português (ex: "Adiciona implementação da fila encadeada", "Corrige colisão de agentes na triagem"). Commits com mensagens genéricas como "ajustes", "teste" ou "versão final" serão penalizados.

Geração do Histórico de Commits

Antes de empacotar o projeto para a entrega, o grupo deve abrir o terminal do sistema na pasta raiz do repositório Git e executar o comando de exportação de logs:

```
git log --oneline > historico_commits.txt
```

O arquivo de texto gerado não deve ter seu conteúdo modificado. O grupo apenas deve adicionar no início ou no fim do arquivo um cabeçalho identificando os integrantes da equipe e suas respectivas contas do GitHub, conforme o modelo abaixo:

INTEGRANTES E USUÁRIOS DO GITHUB

Ana Souza - @anasouza

Bruno Silva - @brunosilva

Carlos Lima - @carloslima

HISTÓRICO DE COMMITS

[Conteúdo gerado pelo `git log --oneline`]

Arquivos e Formatos de Entrega

A entrega do projeto deverá ser realizada exclusivamente pelo ambiente virtual de aprendizagem, contendo os seguintes arquivos:

1. **Código do Projeto (.zip):** Pasta do projeto Processing compactada exclusivamente em formato .zip. O arquivo compactado deve conter a pasta principal do projeto (cujo nome deve ser idêntico ao arquivo principal .pde), contendo todos os arquivos de código fonte, a pasta data com os subdiretórios de mapas e sprites, o arquivo README.md com a tabela de integrantes e o arquivo historico_commits.txt.
2. **Apresentação em PDF:** Slides da apresentação da equipe exportados obrigatoriamente em formato PDF.
3. **Vídeo de Demonstração:** Um vídeo gravado de até 5 minutos demonstrando a simulação em funcionamento estável, com áudio explicativo dos integrantes destacando a inicialização, a dinâmica das filas, a triagem e o reset do sistema.

Critérios Detalhados de Avaliação e Pontuação

O trabalho prático possui valor total de **15,00 pontos**, distribuídos rigorosamente de acordo com os critérios estabelecidos abaixo:

Apresentação Oral (Valor: 5,00 pontos)

A equipe terá entre 8 e 12 minutos para apresentar o projeto. A apresentação não deve se resumir à leitura de linhas de código, devendo focar na arquitetura e na lógica adotada.

- **Introdução e Fluxo Global (0,50 ponto):** Clareza ao explicar o fluxo completo do paciente e a modelagem do hospital no grid.
- **Classe Paciente e Máquina de Estados (0,75 ponto):** Explicação detalhada de como os estados do paciente impedem comportamentos inválidos na simulação.
- **Geração de Tempos Aleatórios (0,75 ponto):** Domínio técnico sobre a implementação e parametrização das distribuições exponencial e normal no projeto.
- **Interfaces das Estruturas de Dados (1,00 ponto):** Clareza em expor as assinaturas e métodos das classes de Lista, Fila e Árvore criadas, justificando suas complexidades.

- **Algoritmo Wavefront e Movimentação Multiagente (0,75 ponto):** Explicação da propagação do Wavefront e da resolução matemática de conflitos de posição.
- **Demonstração Prática (0,75 ponto):** Execução do simulador em tempo real, demonstrando o funcionamento de todas as etapas de atendimento, menus e o comportamento dinâmico das filas.
- **Organização e Controle de Tempo (0,50 ponto):** Postura da equipe, clareza nas respostas aos questionamentos e respeito estrito ao tempo limite estipulado.

Implementação do Código - Funcionalidades (Valor: 6,00 pontos)

Avaliação da corretude algorítmica, boas práticas de orientação a objetos e cumprimento das restrições de código.

- **Leitura e Validação de Mapas (0,50 ponto):** Carregamento dinâmico via menu, detecção de erros de formatação nos arquivos de mapa e validação de caminhos transitáveis.
- **Classe Paciente e Lista Encadeada Autônoma (0,75 ponto):** Implementação robusta da lista de agentes ativos, garantindo inserções e deleções seguras sem perda de referências.
- **Totem, Senhas e Sistema de Assentos (0,75 ponto):** Funcionamento do totem na geração de códigos dinâmicos e controle correto de reserva física de assentos no grid.
- **Triagem e Árvore de Manchester no Vetor (1,00 ponto):** Implementação correta da árvore binária navegada por índices matemáticos de heap dentro do vetor, e lógica de chamada prioritária na triagem.
- **Consultas Médicas e Descarte de Agentes (0,75 ponto):** Funcionamento correto dos consultórios médicos, respeito estrito às prioridades clínicas e eliminação física e lógica dos pacientes no bloco removedor.
- **Algoritmo de Movimento Multiagente e Conflitos (1,50 ponto):** Implementação sem falhas do Wavefront, com tomada de decisões em duas fases (intenção/resolução) impedindo sobreposição de personagens e travamentos lógicos (*deadlocks*).
- **Ordenação de Assentos por Distância (0,50 ponto):** Aplicação de um algoritmo de ordenação clássico codificado manualmente para destinar o paciente ao assento mais próximo utilizando distâncias de caminho reais.
- **Estabilidade Geral e Integração (0,25 ponto):** Execução harmônica de todas as partes integradas sem apresentar vazamentos de memória ou travamentos de personagens ao longo do tempo.

Identidade Visual e Sprites (Valor: 2,00 pontos)

- **Visualização dos Elementos (1,00 ponto):** Uso adequado de sprites gráficos para representar todos os componentes do cenário e os profissionais.

- **Sinalização de Estado e Prioridade (1,00 ponto):** Facilidade de identificação visual da prioridade do paciente (normal/preferencial) e da cor atribuída após a triagem de Manchester, utilizando elementos visuais sobrepostos ou sprites distintos de forma clara.

Sistema de Navegação - Menus (Valor: 2,00 pontos)

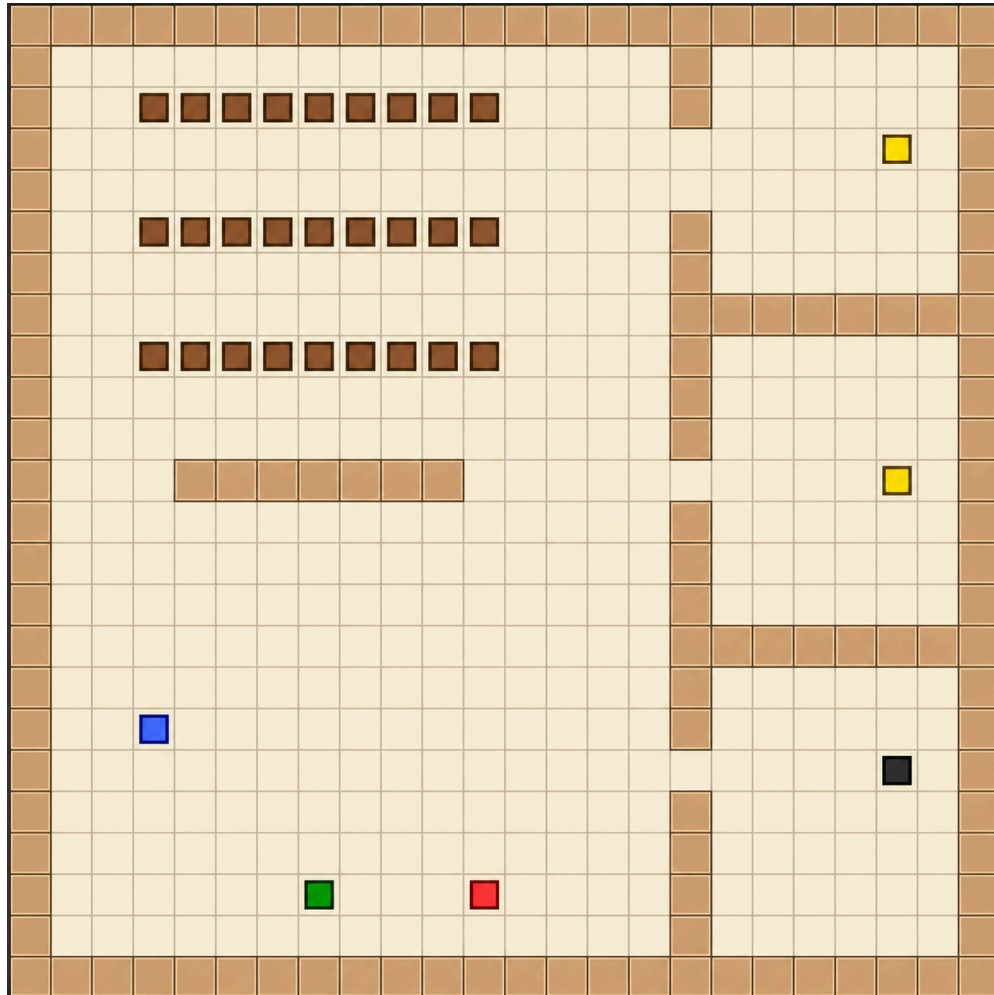
- **Menu Inicial (1,00 ponto):** Seleção interativa de mapas que altera a inicialização física da simulação de forma dinâmica.
- **Menu de Pausa e Reset (1,00 ponto):** Interrupção segura dos tempos físicos e rotina de reset que limpa e reinicializa perfeitamente todos os dados da memória, retornando o sistema ao seu estado original.

Penalidades Aplicáveis à Nota Final

As seguintes deduções de pontos serão efetuadas diretamente na nota consolidada da equipe caso ocorram falhas nas entregas obrigatórias:

- Não envio da apresentação dos slides em formato PDF: **-2,00 pontos.**
- Não envio do vídeo de demonstração prática: **-2,00 pontos.**
- Uso de coleções prontas do Java (como ArrayList, LinkedList, Queue ou HashMap) nas estruturas obrigatórias: **Redução de até 50% da nota atribuída à respectiva funcionalidade.**

Mapa de Exemplo



arquivo do mapa

24 24

```
#####
#.....#.....#
#..AAAAAAAAA...#.....#
#.....M.#
#.....#.....#
#..AAAAAAAAA...#.....#
#.....#.....#
#.....#####
#..AAAAAAAAA...#.....#
#.....#.....#
#.....#.....#
```

```
#...#####.....M.#
#.....#.....#
#.....#.....#
#.....#.....#
#.....#####
#.....#.....#
#..T.....#.....#
#.....E.#
#.....#.....#
#.....#.....#
#.....G...R...#.....#
#.....#.....#
#####
```